



Software Requirements Specification

for

Virtual Fitness Trainer

Version 2.0

Prepared by

Muhammad Wasif (NUM-BSCS-2024-59)

Muhammad Shahid (NUM-BSCS-2024-56)

Hifza Bibi (NUM-BSCS-2024-27)

Namal University, Mianwali
Department of Computer Science

December 28, 2025

Prepared for

Mr. Mutahar Khadim

Research Associate at Center of AI and Big Data

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions, Acronyms, and Abbreviations	4
1.4	Overview	5
2	Overall Description	6
2.1	Product Perspective	6
2.1.1	System Interfaces	6
2.1.2	User Interfaces	6
2.1.3	Hardware Interfaces	6
2.1.4	Software Interfaces	6
2.1.5	Communications Interfaces	7
2.1.6	Operations	7
2.2	Product Functions	7
2.3	User Characteristics	8
2.4	Constraints	8
2.5	Assumptions and Dependencies	9
3	Specific Requirements	10
3.1	Functional Requirements	10
3.1.1	FR-1: User Registration and Profile Creation	10
3.1.2	FR-2: User Authentication and Login	12
3.1.3	FR-3: AI-Based Workout Plan Generation	13
3.1.4	FR-4: AI-Based Diet Plan Generation	15
3.1.5	FR-5: Workout Video Demonstration	17
3.1.6	FR-6: Daily Progress Tracking	18
3.1.7	FR-7: Weekly and Monthly Progress Reports	19
3.1.8	FR-8: Adaptive Plan Recommendations	20
3.1.9	FR-9: User Profile Management	22
3.2	External Interface Requirements	22
3.2.1	User Interfaces	22
3.2.2	Hardware Interfaces	23
3.2.3	Software Interfaces	23
3.2.4	Communications Interfaces	24
3.3	Non-Functional Requirements	24
3.3.1	Reaction Speed and Processing Capacity	24
3.3.2	Security Requirements	25
3.3.3	Reliability Requirements	25
3.3.4	Maintainability Requirements	25
3.3.5	Usability Requirements	26
3.4	Design Constraints	26
3.4.1	Standards Compliance	26
3.4.2	Hardware Limitations	27
3.4.3	Technology Stack Constraints	27

3.5	System Attributes	27
3.5.1	Availability	27
3.5.2	Portability	27
3.5.3	Scalability	27
3.6	Other Requirements	27
3.6.1	Database Requirements	27
3.6.2	Operations	28
3.6.3	AI Model Requirements	29
3.6.4	Legal and Regulatory Requirements	29

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete description of all the functions and specifications of the Virtual Fitness Trainer system. This document is intended for both the stakeholders and the developers of the system.

The purpose of this document is to:

- Define the functional and non-functional requirements of the Virtual Fitness Trainer system.
- Provide a reference for the validation and verification of the system.
- Facilitate the transfer of the software product to new users and maintainers.
- Serve as a baseline for future enhancements and modifications.

The intended audience for this document includes:

- Project managers who need to understand the scope and requirements.
- Software developers who will implement the system.
- Testers who will validate the system.
- Requirement Provider and other stakeholders.
- Documentation writers who will create user manuals.

1.2 Scope

Virtual Fitness Trainer is a comprehensive fitness guidance system designed to provide personalized workout and diet plans to users based on their individual health conditions, fitness goals, and physical characteristics. The system will address the growing need for accessible, customized fitness guidance that adapts to users' health conditions and specific requirements.

The software product will be called **Virtual Fitness Trainer**. The system will enable users to:

- Register and create personalized profiles with health information and physical characteristics.
- Receive AI-generated workout plans tailored to their fitness goals.
- Get customized diet plans based on their health conditions and nutritional needs.
- Track daily and weekly fitness progress.
- View workout video demonstrations.
- Receive recommendations based on their performance.

The major benefits of the Virtual Fitness Trainer system include:

- **Personalization:** Unlike generic fitness apps, this system provides plans according to individual health conditions, age, gender, weight, height, and fitness goals.
- **Accessibility:** Free of cost fitness guidance available to anyone.
- **Health-Conscious:** Special consideration for users with health issues or medical conditions.
- **Motivation:** Progress tracking features to keep users engaged and motivated.
- **Educational:** Workout video demonstrations to ensure proper form and method for workouts.

The goals of this software product are to:

- Provide personalized fitness solutions to users regardless of their current fitness level.
- Increase user engagement by giving recommendations and showing a progress report.
- Ensure user safety by considering health conditions in plan generation.
- Use AI and ML technology to continuously improve recommendations for diets and workouts.
- Create a user-friendly interface for easy to use.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
IEEE	Institute of Electrical and Electronics Engineers
RP	Requirement Provider
UI	User Interface
API	Application Programming Interface
BMI	Body Mass Index - a measure of body fat based on height and weight
AI	Artificial Intelligence
ML	Machine Learning
NFR	Non-Functional Requirement
FR	Functional Requirement
Workout Plan	A structured schedule of exercises to achieve specific fitness goals
Diet Plan	A customized program of meals and caloric intake for the users
Progress Tracking	The system feature that monitors and displays user fitness improvements over time
Workout Video Demonstration	Visual demonstration of exercises showing proper form and method for workouts
User Profile	Collection of user information including health data, and fitness goals

References

- [1] M. D. Mifflin *et al.*, “A new predictive equation for resting energy expenditure in healthy individuals,” *American Journal of Clinical Nutrition*, vol. 51, no. 2, pp. 241-247, Feb. 1990.
- [2] IEEE Std 830-1984, IEEE Guide to Software Requirements Specifications
- [3] Project Proposal: Virtual Fitness Trainer, November 9, 2025
- [4] BodBot, “BodBot Personal Trainer,” Google Play Store, 2025
- [5] Leap Fitness Group, “Home Workout – No Equipment,” Google Play Store, 2025
- [6] Anthropic, Claude,” [Large language model], 2025. Available: <https://www.anthropic.com>
- [7] xAI, Grok,” [Large language model], 2025. Available: <https://x.ai>

1.4 Overview

The remainder of this document is organized as follows:

- **Section 2 - Overall Description:** This section provides a general overview of the Virtual Fitness Trainer system, including product perspective, product functions, user characteristics, constraints, assumptions, and dependencies.
- **Section 3 - Specific Requirements:** This section contains detailed descriptions of all functional and non-functional requirements, external interface requirements, and other specifications necessary for system development. This includes 9 functional requirements covering user registration, AI-based plan generation, workout video demonstrations, progress tracking, and related features.
- **Appendices:** This section include Context Diagram, Use Case Diagram, and traceability matrix for requirement tracking.

2 Overall Description

2.1 Product Perspective

Virtual Fitness Trainer is an independent, self-contained fitness guidance system that operates as a complete solution for personalized fitness planning and tracking. The system will be a web-based platform.

2.1.1 System Interfaces

The Virtual Fitness Trainer system does not interface with external systems but operates as a standalone platform. However, it may integrate with:

- Cloud storage services for backup and data synchronization
- Email services for user notifications and password recovery
- Analytics services for monitoring system usage and performance

2.1.2 User Interfaces

The system will provide the following user interfaces:

- **Web Application Interface:** Python-based web interface offering similar functionality accessible through web browsers
- **Dashboard Interface:** Central user dashboard displayed after login, providing access to plan generation forms, progress overview, and key actions
- **Workout Video Demonstration Interface:** Visual demonstration module showing video-based exercise demonstrations

All interfaces will follow responsive design principles and accessibility standards to ensure usability across different devices and for users with disabilities.

2.1.3 Hardware Interfaces

The system will interact with the following hardware:

- **Network Interface:** internet connection for cloud synchronization

2.1.4 Software Interfaces

The system will interact with the following software components:

- **Operating System:** Windows, macOS, Linux for web access.
- **Database:** MongoDB 5.0+ for data storage and retrieval.
- **Backend Framework:** Node.js 16+ with Express.js 4.x.
- **Machine Learning Framework:** TensorFlow or PyTorch for AI model implementation.
- **Authentication Service:** JWT-based authentication system.

2.1.5 Communications Interfaces

- **Network Protocols:** HTTPS for secure communication between client and server.
- **API Architecture:** RESTful API following industry standards.
- **Data Formats:** JSON for data exchange between frontend and backend.
- **Real-time Communication:** WebSocket for potential future real-time features.

2.1.6 Operations

- **User-initiated Operations:** Registration, login, profile updates, plan generation, progress logging
- **Automated Operations:** Daily plan notifications, weekly progress reports, plan regeneration based on progress

2.2 Product Functions

The major functions that Virtual Fitness Trainer will perform include:

1. User Management and Authentication:

- User registration with detailed profile creation.
- Secure login and authentication.
- Profile management and updates.
- Password recovery functionality.

2. AI-Powered Plan Generation:

- Generate personalized workout plans using trained AI models.
- Create customized diet plans based on nutritional requirements.
- Adapt plans based on health conditions and constraints.
- Consider user goals (weight loss, muscle gain, maintenance).

3. Workout Video Demonstration:

- Display video demonstrations of exercises.
- Provide step-by-step instructions for each workout.
- Show proper form and technique.
- Include difficulty levels and variations.

4. Progress Tracking and Analytics:

- Track daily workout completion.
- Monitor weight and measurement changes.
- Generate weekly and monthly progress reports.
- Visualize progress through charts and graphs.

5. Adaptive Recommendations:

- Adjust plans based on user performance.
- Suggest alternative exercises for health conditions.
- Provide motivational tips and guidance.
- Recommend plan modifications based on progress.

2.3 User Characteristics

The expected users of Virtual Fitness Trainer include:

User Type	Characteristics
Fitness Beginners	Technical expertise: Basic computer usage, Education: High school or above, Experience: No prior fitness software experience required, Age: 18-65 years
Health-Conscious Individuals	Technical expertise: Moderate, Education: Varies, Experience: May have used other fitness softwares, Special needs: Users with health conditions requiring modified exercises
Fitness Enthusiasts	Technical expertise: Moderate to High, Education: Varies, Experience: Familiar with fitness terminology and apps, Goals: Advanced workout plans and detailed tracking
Fitness Trainers	Technical expertise: Moderate, Education: Certified fitness trainers or healthcare professionals, Experience: Professional fitness guidance, Role: Plan validation and content review

2.4 Constraints

The following constraints apply to the development of this system:

1. **Regulatory Policies:** The system must follow the health data privacy regulations and ensure user data protection according to applicable laws.
2. **Interface Requirements:** The system must provide an easy to use interfaces suitable for users with different technical expertise of users.
3. **Parallel Operations:** The system must support multiple users accessing workout plans and tracking progress at a time.
4. **Audit Functions:** All user activities and plan generations must be logged for quality assurance and system improvement.
5. **Higher-Order Language Requirements:** Development must use Python for the web interface, and Node.js for backend services.
6. **Reliability Requirements:** The system must maintain high availability with minimum downtime for plan access and progress tracking.

7. **Criticality of Application:** As the system provides health-related guidance, it must provide accurate recommendations.
8. **Safety and Security Considerations:** User health data must be encrypted and stored safely. The system must include warnings for users with serious health conditions to consult healthcare professionals.

2.5 Assumptions and Dependencies

The following assumptions and dependencies have been made during the preparation of this SRS:

Assumptions:

1. Users have access to computers or devices with web browsers and internet connection.
2. Users will provide accurate information during registration for effective plan generation.
3. Users understand basic fitness terminology or will learn through provided guidance.
4. The AI model training data includes different fitness plans suitable for various health conditions.
5. Users will consult healthcare professionals for serious health conditions before starting any fitness program.

Dependencies:

1. Availability of MongoDB cloud hosting services for database management.
2. Reliable deployment platform (Vercel) for backend hosting.
3. Access to quality training data for AI model development.
4. Availability of workout demonstration videos.
5. Continuous internet connection for real-time plan generation.

3 Specific Requirements

3.1 Functional Requirements

This section describes the functional requirements of the Virtual Fitness Trainer system.

3.1.1 FR-1: User Registration and Profile Creation

Introduction This function enables new users to register in the system and create profiles. The registration process collects essential user information, including demographics, health data, and fitness goals. This information serves as the foundation for generating personalized workout and diet plans.

Inputs

- **Personal Information:** Name (string, 2-50 characters), Email (valid email format), Password (minimum 8 characters with uppercase, lowercase, and numbers), Date of Birth (date format, e.g., YYYY-MM-DD, must indicate user is at least 18 years old), Gender (Male/Female/Other).
- **Physical Measurements:** Height (numeric, in cm or feet/inches, range 120-250 cm), Weight (numeric, in kg or lbs, range 30-300 kg), Current Body Type (Slim/Average/Athletic/Overweight).
- **Health Information:** Existing health conditions (multiple selection: diabetes, hypertension, joint problems, heart conditions, asthma, other, none), Injuries or physical limitations (text field, optional), Medications (text field, optional).
- **Fitness Goals:** Primary goal (Weight Loss/Muscle Gain/Maintain Fitness/Improve Endurance/General Health), Target weight (numeric, optional), Timeline (1-12 months).
- **Activity Level:** Current fitness level (Beginner/Intermediate/Advanced), Preferred workout frequency (1-7 days per week).

Processing: The system shall perform the following operations:

1. Validate all input fields for correct format and acceptable ranges.
2. Check email uniqueness in the database to prevent duplicate accounts.
3. Hash and securely store the password using bcrypt or similar encryption.
4. Calculate user's age from date of birth using current date minus birth date.
5. Validate that calculated age is at least 18 years.
6. Calculate BMI from height and weight: $BMI = \text{weight}(\text{kg}) / [\text{height}(\text{m})]^2$.
7. Store all user information in the MongoDB database.
8. Generate a unique user ID for database reference.
9. Create an initial user profile document with all provided information.

10. Send a verification email to the provided email address.

Validity checks:

- Email format validation using regex pattern.
- Password strength validation (minimum 8 characters, mixed case, numbers).
- Calculated age at least 18 years.
- Height and weight within realistic human ranges.
- All mandatory fields must be completed.

Error handling:

- **Duplicate Email:** Display message "This email is already registered. Please use a different email or login."
- **Weak Password:** Display message "Password must be at least 8 characters with uppercase, lowercase, and numbers."
- **Invalid Input Range:** Display message "Please enter a valid value for [field name] within the acceptable range."
- **Underage User:** Display message "You must be at least 18 years old to register."
- **Database Connection Error:** Display message "Registration failed. Please try again later."

Outputs

- **Success Confirmation:** Registration success message with user ID, displayed on screen and sent via email
- **User Profile Document:** Complete user profile stored in database with timestamp
- **Verification Email:** Email containing account verification link sent to user's email address
- **Navigation:** Automatic redirect to profile completion or dashboard page after successful registration

Requirement Statement FR-1.1: The system shall allow users to register by providing name, email, password, date of birth, gender, height, weight, health conditions, and fitness goals.

FR-1.2: The system shall validate all registration inputs and ensure email uniqueness before account creation.

FR-1.3: The system shall securely store user passwords using encryption and never store passwords in plain text.

FR-1.4: The system shall calculate age from date of birth and store the user's BMI during registration for future reference.

FR-1.5: The system shall send a verification email to the user after successful registration.

3.1.2 FR-2: User Authentication and Login

Introduction This function provides secure access to the system through user authentication, including password recovery. Users must verify their identity using registered credentials to access personalized features, workout plans, and progress tracking.

Inputs

- **Email:** Registered email address (string, valid email format)
- **Password:** User password (string, minimum 8 characters)
- **Remember Me:** Optional checkbox for persistent login (boolean)
- **Forgot Password Initiation:** Email address for password reset request (optional)

Processing The system shall:

1. Accept email and password from the login form.
2. Query the database for a user record matching the provided email.
3. Compare the provided password with the stored encrypted password using bcrypt.
4. Generate JWT (JSON Web Token) upon successful authentication.
5. Set session expiration based on "Remember Me" option (7 days if checked, 24 hours otherwise).
6. Log the login activity with timestamp and device information.
7. Increment failed login attempts counter if authentication fails.
8. Lock account temporarily after 5 consecutive failed login attempts.
9. Redirect user to dashboard upon successful login.
10. For forgot password: Verify email exists, generate secure one-time reset token with 1-hour expiry, store token, send reset link via email.
11. On reset link access: Validate token and expiry, allow entry of new password, update encrypted password, invalidate token.

Validity checks:

- Email exists in the database.
- Account is active and not suspended.
- Account is not temporarily locked due to failed attempts.
- Password matches stored encrypted password.
- Reset token is valid and not expired.

Error handling:

- **Invalid Credentials:** Display "Incorrect email or password. Please try again."
- **Account Locked:** Display "Account temporarily locked due to multiple failed attempts. Please try again in 30 minutes or reset your password."
- **Unverified Email:** Display "Please verify your email address before logging in. Check your inbox for the verification link."
- **Invalid/Expired Reset Token:** Display "The password reset link is invalid or has expired. Please request a new one."
- **Non-existent Email for Reset:** Display "No account found with that email address."

Outputs

- **Authentication Token:** JWT token stored in client storage for subsequent API calls.
- **User Session:** Active session created with expiration time.
- **Dashboard Access:** Automatic redirect to the user dashboard showing personalized content.
- **Login Log:** Entry in system logs recording successful/failed login attempt.
- **Password Reset Email:** Email containing time-limited reset link.
- **Password Update Confirmation:** "Password successfully updated. Please log in with your new password."

Requirement Statement FR-2.1: The system shall authenticate users using email and password credentials.

FR-2.2: The system shall generate and provide a secure JWT token upon successful authentication for subsequent API requests.

FR-2.3: The system shall lock user accounts temporarily after 5 consecutive failed login attempts within 30 minutes.

FR-2.4: The system shall maintain login session for 24 hours by default or 7 days if "Remember Me" is selected.

FR-2.5: The system shall provide a "Forgot Password" option on the login screen.

FR-2.6: The system shall send a password reset link via email that is valid for 1 hour.

FR-2.7: The system shall allow users to set a new password after verifying the reset link.

3.1.3 FR-3: AI-Based Workout Plan Generation

Introduction This function uses artificial intelligence and machine learning to generate personalized workout plans. Preferences are collected via a dedicated form accessed from the dashboard. The AI model considers user profile data, fitness goals, health conditions, and current fitness level to create safe and effective exercise routines.

Inputs

- **User Profile Data:** Date of birth (for age calculation), gender, weight, height, BMI (retrieved from database)
- **Health Conditions:** List of medical conditions and physical limitations
- **Fitness Goal:** Weight loss, muscle gain, endurance, or general fitness
- **Fitness Level:** Beginner, intermediate, or advanced
- **Available Days:** Number of workout days per week (1-7)
- **Session Duration:** Preferred workout duration (15, 30, 45, or 60 minutes)
- **Equipment Access:** Home workout (no equipment) or gym access
- **Target Areas:** Optional body parts to focus on (arms, legs, core, full body)

Processing The system shall:

1. Retrieve complete user profile from database.
2. Prepare input features for AI model (normalize age, weight, height, encode categorical variables).
3. Load trained machine learning model from model repository.
4. Feed user characteristics into the AI model.
5. Generate exercise recommendations based on model predictions.
6. Filter exercises based on health conditions (exclude high-impact exercises for joint problems, avoid heavy lifting for heart conditions).
7. Adjust exercise intensity based on fitness level.
8. Create a weekly schedule distributing exercises across specified workout days.
9. Calculate estimated calorie burn for each workout session.
10. Assign rest days to prevent overtraining.
11. Include warm-up and cool-down exercises.
12. Store generated workout plan in database with timestamp.
13. Link the workout plan to the user profile for retrieval.

Validity checks:

- User profile exists and is complete.
- All required fields for plan generation are available.
- Health conditions are properly coded for model input.

- Generated exercises are appropriate for the user's fitness level.
- Workout duration matches user preference.

Error handling:

- **Incomplete Profile:** Display "Please complete your profile before generating a workout plan."
- **Model Loading Error:** Display "Unable to generate plan. Please try again later." and log error for review
- **Invalid Input Data:** Display "Some profile information is invalid. Please update your profile."

Outputs

- **Workout Plan Document:** Structured document containing:
 - Weekly schedule (7 days)
 - Daily exercise list with sets, reps, and duration
 - Exercise difficulty level
 - Estimated time for each workout
 - Total weekly calorie burn estimate
 - Rest days
- **Plan Summary:** Overview displayed on dashboard showing plan highlights
- **PDF Export:** Option to download plan as PDF for offline reference
- **Database Record:** Workout plan saved with user ID, creation date, and plan version

Requirement Statement FR-3.1: The system shall use a trained AI/ML model to generate personalized workout plans based on user age, gender, weight, height, fitness goals, and health conditions.

FR-3.2: The system shall exclude exercises that are not suitable for users' specific health conditions.

FR-3.3: The system shall create weekly workout schedules with appropriate rest days based on the user's selected workout frequency.

FR-3.4: The system shall calculate and display the estimated calorie burn for each workout session.

FR-3.5: The system shall allow users to regenerate workout plans if they are unsatisfied with initial recommendations.

FR-3.6: The system shall provide a dedicated form accessible from the dashboard for users to input workout preferences before plan generation.

3.1.4 FR-4: AI-Based Diet Plan Generation

Introduction This function generates personalized diet plans using AI algorithms trained on nutritional data and dietary guidelines. Preferences are collected via a dedicated form accessed from the dashboard. The diet plan considers the user's metabolic rate, fitness goals, health conditions, dietary restrictions, and cultural preferences.

Inputs

- **User Profile:** Date of birth (for age), gender, weight, height, activity level
- **Fitness Goal:** Weight loss, muscle gain, or maintenance
- **Dietary Preferences:** Vegetarian, vegan, non-vegetarian, pescatarian
- **Food Allergies:** List of allergies (nuts, dairy, gluten, shellfish, etc.)
- **Cultural Preferences:** Pakistani/South Asian, Western, etc.
- **Meal Frequency:** Number of meals per day (3-6 meals)
- **Budget Level:** Low, medium, or high budget for meals
- **Cooking Capability:** Complex recipes, simple meals, or ready-to-eat preference

Processing The system shall:

1. Calculate Basal Metabolic Rate (BMR) using the Mifflin-St Jeor equation [1]:
 - Men: $BMR = (10 \times \text{weight in kg}) + (6.25 \times \text{height in cm}) - (5 \times \text{age}) + 5$
 - Women: $BMR = (10 \times \text{weight in kg}) + (6.25 \times \text{height in cm}) - (5 \times \text{age}) - 161$
2. Calculate Total Daily Energy Expenditure (TDEE) = BMR × Activity Factor
3. Adjust calorie target based on goal:
 - Weight loss: TDEE - 500 calories (for 0.5 kg/week loss)
 - Muscle gain: TDEE + 300 calories
 - Maintenance: TDEE
4. Calculate macronutrient distribution (protein, carbs, fats) based on goal.
5. Load trained diet plan generation model.
6. Input user constraints and preferences to model.
7. Generate meal suggestions for breakfast, lunch, dinner, and snacks.
8. Ensure meals meet calorie and macro targets.
9. Filter out allergenic ingredients.
10. Adapt recipes to cultural preferences.
11. Include portion sizes and preparation instructions.
12. Create weekly meal plan with variations.
13. Store diet plan in database linked to user profile.

Validity checks:

- Calorie target is within healthy range (minimum 1200 for women, 1500 for men).
- Macronutrient ratios are balanced and safe.
- All suggested meals exclude specified allergic ingredients.
- Meals align with dietary preferences.

Error handling:

- **Insufficient Data:** Display "Please complete dietary preferences in your profile."
- **Extreme Calorie Deficit:** Display warning "Target calorie level is very low. Consult a nutritionist for safe weight loss."
- **Model Error:** Display "Unable to generate diet plan. Please try again."

Outputs

- **Weekly Meal Plan:** 7-day meal plan with breakfast, lunch, dinner, and snacks.
- **Nutritional Summary:** Daily calorie count, protein, carbs, fat breakdown.
- **Shopping List:** Ingredient list for the week's meals.
- **Recipe Cards:** Detailed recipes with cooking instructions.
- **Portion Guides:** Visual portion size recommendations

Requirement Statement FR-4.1: The system shall calculate user's BMR and TDEE to determine appropriate calorie targets for their fitness goals.

FR-4.2: The system shall use AI model to generate personalized diet plans according to the user's dietary preferences, allergies, and cultural food choices.

FR-4.3: The system shall create balanced meal plans with appropriate macronutrient distribution (protein, carbohydrates, fats).

FR-4.4: The system shall generate weekly shopping lists based on the created diet plan.

FR-4.5: The system shall provide recipe instructions and portion size guidance for all recommended meals.

FR-4.6: The system shall provide a dedicated form accessible from the dashboard for users to input diet preferences before plan generation.

3.1.5 FR-5: Workout Video Demonstration

Introduction This function provides visual demonstrations of exercises through videos. It helps users understand proper form and method of workout to prevent injuries and maximize effectiveness.

Inputs

- **Exercise Selection:** User selects an exercise from their workout plan.
- **Exercise ID:** Unique identifier for the exercise in database.
- **User Preference:** Video demonstration preference.

Processing The system shall:

1. Retrieve exercise details from database using exercise ID.
2. Load corresponding demonstration video.
3. Display exercise name and muscle groups targeted.
4. Show step-by-step instructions.
5. Play visual demonstration with proper form.
6. Highlight common mistakes to avoid.
7. Display recommended sets, reps, and rest periods.
8. Provide alternative variations for different fitness levels.
9. Allow pause, replay, and slow-motion viewing.
10. Track which exercises has been viewed by the user.

Outputs

- **Video Player:** Media player showing exercise demonstration.
- **Exercise Instructions:** Text description of exercise execution.
- **Form Tips:** List of key points for proper form.
- **Variations Display:** Easier or harder versions of the exercise.
- **Tracking Record:** Log of viewed demonstrations.

Requirement Statement FR-5.1: The system shall provide video demonstrations for every exercise in the workout plan.

FR-5.2: The system shall display step-by-step instructions along with video demonstrations.

FR-5.3: The system shall show common mistakes and proper form tips for each exercise.

FR-5.4: The system shall allow users to pause, replay, and view demonstrations in slow motion.

3.1.6 FR-6: Daily Progress Tracking

Introduction This function enables users to log their daily workout completion, weight measurements, and other fitness metrics. Consistent tracking helps users monitor improvements and keeps them motivated.

Inputs

- **Workout Completion:** Checkbox for each completed exercise.
- **Weight Entry:** Daily weight measurement (kg or lbs).
- **Body Measurements:** Chest, waist, hips, arms, thighs (cm or inches).
- **Energy Level:** Rating scale 1-5.
- **Notes:** Optional text field for observations.
- **Date:** Date of entry (auto-populated with current date).

Processing The system shall:

1. Validate date to ensure it's not in the future.
2. Check if entry already exists for the date.
3. Store workout completion status for each exercise.
4. Record weight and body measurements.
5. Calculate changes from previous entries.
6. Update running totals (total workouts completed, current streak).
7. Check if user met weekly workout goal.
8. Store data in user's progress collection.
9. Trigger achievement notifications if milestones reached.

Outputs

- **Confirmation Message:** "Progress logged successfully for [date]".
- **Progress Summary:** Display of logged data.
- **Trend Indicators:** Up/down arrows showing changes from previous entry.
- **Streak Counter:** Number of consecutive days with logged workouts.

Requirement Statement FR-6.1: The system shall allow users to mark completed exercises on a daily basis.

FR-6.2: The system shall enable users to log weight and body measurements daily.

FR-6.3: The system shall calculate and display changes from previous entries.

FR-6.4: The system shall maintain a workout streak counter showing consecutive days of completed workouts.

3.1.7 FR-7: Weekly and Monthly Progress Reports

Introduction This function generates progress reports summarizing user progress over weekly and monthly periods. Reports provide the details of workout consistency, weight changes, and goal achievement.

Inputs

- **Report Type:** Weekly or monthly
- **Date Range:** Start and end dates for the report
- **User ID:** Authenticated user requesting report

Processing

 The system shall:

1. Check database for all progress entries within date range
2. Calculate total workouts completed
3. Calculate average daily workout completion rate
4. Analyze weight change (start weight - end weight)
5. Calculate average weekly weight loss/gain
6. Identify most and least completed exercises
7. Determine consistency score based on scheduled vs. completed workouts
8. Generate charts showing weight trends
9. Create graphs for body measurement changes
10. Compare current period with previous period
11. Generate insights and recommendations

Outputs

- **Report Dashboard:** Visual summary with charts and graphs
- **Statistics:** Total workouts, completion rate, weight change
- **Trend Charts:** Line graphs showing progress over time
- **Insights:** AI-generated observations and recommendations
- **PDF Export:** Downloadable progress report

Requirement Statement FR-7.1: The system shall generate weekly progress reports showing workout completion rate and weight changes.

FR-7.2: The system shall generate monthly progress reports with detailed analytics and trend visualization.

FR-7.3: The system shall compare current period performance with previous periods.

FR-7.4: The system shall allow users to export progress reports as PDF files.

3.1.8 FR-8: Adaptive Plan Recommendations

Introduction This function analyzes user progress and automatically suggests plan modifications.

Inputs

- **Progress Data:** Historical workout completion and weight changes
- **Current Plan:** Active workout and diet plan details
- **Time Period:** Duration since plan started
- **User Feedback:** Optional difficulty ratings

Processing The system shall:

1. Analyze progress data for the past 4 weeks
2. Calculate rate of progress toward goal
3. Identify if user is:
 - On track (expected progress)
 - Ahead of schedule (exceeding expectations)
 - Behind schedule (slower than expected progress)
 - Plateauing (no progress for 2+ weeks)
4. For plateaus: recommend increasing intensity or changing exercises
5. For ahead-of-schedule: suggest more challenging variations
6. For behind-schedule: check consistency, suggest easier variations or fewer days
7. Generate recommendation with reasoning
8. Present options to user for plan modification

Outputs

- **Recommendation Card:** Summary of suggested changes
- **Reasoning:** Explanation based on progress analysis
- **Preview:** Show what new plan would look like
- **Action Buttons:** Accept, modify, or dismiss recommendation

Requirement Statement FR-8.1: The system shall analyze user progress every 4 weeks and identify plateaus or exceptional progress.

FR-8.2: The system shall automatically recommend plan modifications when progress stalls for more than 2 weeks.

FR-8.3: The system shall suggest increased difficulty for users consistently exceeding workout targets.

FR-8.4: The system shall allow users to accept, modify, or dismiss adaptive recommendations.

3.1.9 FR-9: User Profile Management

Introduction This function allows users to view and update their profile information, including personal details, health conditions, fitness goals, and preferences.

Inputs

- **Updated Fields:** Any profile field user wishes to change
- **Verification:** Password confirmation for sensitive changes

Processing The system shall:

1. Display current profile information
2. Allow editing of modifiable fields
3. Validate updated information
4. Check if changes require plan regeneration
5. Update database with new information
6. Log profile changes with timestamp
7. Notify user if workout/diet plans need updating
8. Trigger plan regeneration if significant changes made

Outputs

- **Update Confirmation:** "Profile updated successfully"
- **Impact Notice:** Notification if plans need regeneration
- **Updated Profile Display:** Show new information

Requirement Statement FR-9.1: The system shall allow users to view and edit their profile information at any time.

FR-9.2: The system shall notify users when profile changes require regenerating workout or diet plans.

FR-9.3: The system shall validate updated information before saving changes.

FR-9.4: The system shall maintain a log of profile changes for audit purposes.

3.2 External Interface Requirements

3.2.1 User Interfaces

The system shall provide the following user interfaces:

1. **UI-2: Web Application Interface**
 - Screen format: Responsive web design for desktop and tablet browsers

- Layout: Side navigation panel with content area
- Navigation: Breadcrumb navigation and dropdown menus
- Compatibility: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+

2. UI-3: Dashboard Interface

- Screen format: Responsive web interface
- Layout: Central hub with buttons for "Generate Workout Plan", "Generate Diet Plan", progress summary, and quick actions
- Features: Personalized greeting, current plan overview, progress widgets

3. UI-4: Workout Video Player

- Screen format: Full-screen or embedded player
- Controls: Play, pause, replay, slow motion, next/previous
- Display: Exercise timer, rep counter, rest period countdown
- Overlay: Text instructions and form tips

3.2.2 Hardware Interfaces

The system has no specific hardware interfaces, as it is a web-based platform that is accessible through web browsers.

3.2.3 Software Interfaces

1. SI-1: The system shall interface with MongoDB Database

- Name: MongoDB
- Version: 5.0 or higher
- Purpose: Store user profiles, workout plans, diet plans, and progress data
- Data format: JSON/BSON documents
- Connection: MongoDB connection string with authentication

2. SI-2: The system shall interface with Node.js Backend

- Name: Node.js with Express.js
- Version: Node.js 16+, Express.js 4.x
- Purpose: Handle API requests, business logic, authentication
- Communication: RESTful API with JSON payloads

3. SI-3: The system shall interface with TensorFlow/PyTorch

- Name: TensorFlow or PyTorch
- Version: TensorFlow 2.x or PyTorch 1.x
- Purpose: Load and execute trained AI models for plan generation
- Integration: Python microservice or TensorFlow.js

4. **SI-4:** The system shall interface with Email Service

- Name: SendGrid or similar SMTP service
- Purpose: Send verification emails, notifications, password resets
- Protocol: SMTP with TLS encryption

3.2.4 **Communications Interfaces**

1. **CI-1:** The system shall use HTTPS protocol (TLS 1.2 or higher) for all client-server communications to ensure data security.
2. **CI-2:** The system shall implement RESTful API architecture with JSON data format for all API endpoints.
3. **CI-3:** The system shall support WebSocket connections for potential real-time features such as live workout sessions.
4. **CI-4:** The system shall use JWT (JSON Web Tokens) for stateless authentication across API requests.

3.3 **Non-Functional Requirements**

3.3.1 **Reaction Speed and Processing Capacity**

NFR-1: Reaction Speed and Processing Capacity

NFR-1.1: The system shall answer user login requests within 2 seconds under normal load conditions (95% of the time).

NFR-1.2: The system shall create AI-based workout plans within 5 seconds of user request submission.

NFR-1.3: The system shall create AI-based diet plans within 7 seconds of user request submission.

NFR-1.4: The system shall load workout video demonstrations within 3 seconds on standard broadband connections (5 Mbps+).

NFR-1.5: The system shall support at least 1000 simultaneous users without slowing down.

NFR-1.6: The system shall handle progress tracking data entry and show confirmation within 1 second.

Static Requirements

- Number of simultaneous users: Minimum 1000 simultaneous users
- Database capacity: Support for 100,000+ user profiles
- Storage: 500 GB minimum for database and media files
- Workout exercise database: Minimum 200 exercises
- Diet recipe database: Minimum 500 recipes

Dynamic Requirements

- API request processing rate: 100 requests per second minimum

- Data synchronization: Real-time sync within 2 seconds
- High-Traffic Handling: Handle 3x normal traffic during peak hours (6-9 AM, 6-9 PM)
- Data Search Speed: 95% of queries complete within 100ms

3.3.2 Security Requirements

NFR-2: System Security and Data Protection

NFR-2.1: The system shall use user permission levels to stop unauthorized access with two roles: User and Fitness Trainer.

NFR-2.2: The system shall scramble all sensitive data including passwords using strong password hashing with minimum 10 salt rounds, and health information using advanced data protection both while sending (HTTPS) and at rest (database scrambling).

NFR-2.3: The system shall keep an activity record of all security-related events including login attempts (successful and failed), profile changes, plan creations, and actions with timestamp and IP address.

NFR-2.4: The system shall automatically lock user accounts for 30 minutes after 5 repeated wrong logins within a 30-minute window.

NFR-2.5: The system shall enforce password rules requiring minimum length of 8 characters including at least one uppercase letter, one lowercase letter, one number, and one special character.

NFR-2.6: The system shall use secure access tokens that expire (24 hours for regular sessions, 7 days for "remember me") and token refresh methods.

NFR-2.7: The system shall follow data privacy rules by allowing users to export their data and request account deletion.

NFR-2.8: The system shall clean all user inputs to prevent database attacks, web script attacks, and other injection attacks.

3.3.3 Reliability Requirements

NFR-3: System Reliability and Availability

NFR-3.1: The system shall maintain an availability time of 99.5% during working hours (calculated monthly, excluding planned maintenance).

NFR-3.2: The system shall have an average time without breakdowns of at least 720 hours (30 days).

NFR-3.3: The system shall automatically perform partial data copies every 6 hours and complete data copies daily at 2:00 AM UTC.

NFR-3.4: The system shall be able to recover from failures with a downtime limit of 30 minutes and data loss limit of 6 hours.

NFR-3.5: The system shall use data duplication with at least one duplicate set for backup and automatic backup switch.

NFR-3.6: The system shall handle API failures smoothly and display user-friendly error messages instead of system errors.

3.3.4 Maintainability Requirements

NFR-4: System Maintainability and Code Quality

NFR-4.1: The system code shall maintain a minimum of 70% code explanation percentage including inline comments, function explanations, and README files.

NFR-4.2: The system structure shall follow building block structure principles with clear divided responsibilities (frontend, backend, database, AI models) to make independent module updates easier.

NFR-4.3: The system shall provide detailed error tracking for finding and fixing problems with log levels (DEBUG, INFO, WARNING, ERROR, CRITICAL).

NFR-4.4: The system shall allow setup changes (API endpoints, database connections, feature flags) through adjustable settings without needing code redeployment.

NFR-4.5: The system shall be designed to allow updates and fixes to be applied with minimal downtime (less than 5 minutes for minor updates).

NFR-4.6: The system codebase shall follow consistent code rules and style guides (ESLint for JavaScript, PEP 8 for Python).

NFR-4.7: The system shall include self-checking code covering at least 60% of backend code and critical frontend components.

3.3.5 Usability Requirements

NFR-5: System Usability and User Experience

NFR-5.1: The system shall enable new users to complete registration and generate their first workout plan within 10 minutes without external help.

NFR-5.2: The system shall provide relevant popup tips and a comprehensive FAQ section accessible from all screens.

NFR-5.3: The system shall display clear and helpful fix suggestions that guide users toward resolution (e.g., "Password must contain at least one uppercase letter" instead of "Invalid password").

NFR-5.4: The system shall be accessible to users with disabilities, implementing accessibility standards including screen reader compatibility, mouse-free controls, and sufficient readable text colors (minimum 4.5:1).

NFR-5.5: The system interface shall be easy to understand, requiring no more than 3 clicks/taps to reach any major function from the home screen.

NFR-5.6: The system shall support multiple languages with initial support for English and Urdu, with the ability to add more languages in future updates.

NFR-5.7: The web interface shall be responsive and adapt to different screen sizes (e.g., desktop, tablet) without loss of functionality.

NFR-5.8: The system shall provide welcome guides for first-time users explaining key features (plan generation, progress tracking, workout video demonstrations).

3.4 Design Constraints

3.4.1 Standards Compliance

1. The system shall comply with IEEE 830-1984 standard for software requirements specifications
2. The system shall follow REST API design principles and best practices
3. The system shall adhere to OWASP Top 10 security standards for web applications
4. The system shall comply with health information privacy guidelines

3.4.2 Hardware Limitations

None.

3.4.3 Technology Stack Constraints

1. Web interface must be developed using Python-based framework
2. Backend must use Node.js with Express.js framework
3. Database must be MongoDB (NoSQL)
4. Version control must use Git with GitHub repository
5. Deployment platform must be Vercel or similar cloud platform

3.5 System Attributes

3.5.1 Availability

The system shall provide 99.5% availability during operational hours, calculated on a monthly basis excluding scheduled maintenance windows (announced 48 hours in advance).

3.5.2 Portability

The system shall be designed to operate on:

- Web browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- Operating systems: Windows 10+, macOS 10.15+, Linux (Ubuntu 18.04+)

3.5.3 Scalability

The system shall be designed to scale horizontally to accommodate growth in users (up to 100,000 users) and data volume through:

- Microservices architecture allowing independent scaling
- Database sharding capabilities for user data distribution
- Content Delivery Network (CDN) for media file delivery
- Load balancing across multiple server instances

3.6 Other Requirements

3.6.1 Database Requirements

1. The system shall use MongoDB 5.0+ for data storage with following collections:
 - Users: User profiles and authentication data
 - WorkoutPlans: Generated workout plans
 - DietPlans: Generated diet plans

- Progress: Daily progress tracking entries
 - Exercises: Exercise library with demonstrations
 - Recipes: Recipe database for diet plans
 - AuditLogs: System activity logs
2. Data retention period: User data retained indefinitely until user requests deletion; audit logs retained for 2 years
 3. Backup frequency: Incremental backups every 6 hours, full backups daily
 4. Database access control: Role-based access with encrypted connections
 5. Data encryption: AES-256 encryption for sensitive fields

3.6.2 Operations

1. Normal operations:

- 24/7 system availability for user access
- Real-time plan generation and progress tracking
- Continuous monitoring of system health metrics

2. Backup procedures:

- Automated daily full backups at 2:00 AM UTC
- Incremental backups every 6 hours
- Backup retention: Daily backups for 30 days, monthly backups for 1 year
- Backup storage: Secure cloud storage with encryption

3. Recovery procedures:

- Automated failover to replica database within 5 minutes
- Manual recovery from backups within 30 minutes
- Data integrity verification after recovery
- User notification of any data recovery operations

4. Maintenance windows:

- Scheduled maintenance: Last Sunday of the month, 2:00-4:00 AM UTC
- Emergency maintenance: As needed with minimal disruption
- Advance notification: 48 hours for scheduled maintenance

3.6.3 AI Model Requirements

1. The AI models for workout and diet plan generation shall be trained on diverse datasets representing various demographics, fitness levels, and health conditions
2. Model accuracy requirements: Minimum 85% user satisfaction with generated plans (measured through feedback)
3. Model retraining: Quarterly updates with new data from user feedback and progress tracking
4. Model versioning: Maintain version control for AI models with the ability to rollback if needed
5. Model explainability: Provide reasoning for plan recommendations to increase user trust

3.6.4 Legal and Regulatory Requirements

1. The system shall display a medical disclaimer stating that it is not a substitute for professional medical advice, and users should consult healthcare providers before starting any fitness program
2. The system shall comply with data protection regulations, including user consent for data collection and processing
3. The system shall provide terms of service and privacy policy accessible during registration
4. The system shall allow users to exercise their data rights: access, portability, and deletion
5. The system shall include age verification to ensure users are 18 years or older